API Security Risk Analysis - JSONPlaceHolder

TASK 03 OF FUTURE INTERNS CYBERSECURITY TRACK

Prepared By: Ellenor Mariez Cauyan | Cybersecurity Intern

CIN ID: FIT/APR26/CS7637

EXECUTIVE SUMMARY

This assessment evaluated the security posture of JSONPlaceholder (a popular, open-source "fake" REST API used by developers for testing and prototyping) to identify vulnerabilities using the OWASP API Security Top 10 framework. The analysis revealed critical gaps in authorization and data exposure, notably High-severity Broken Object Level Authorization (BOLA) and PII leakage, which enable unauthorized access to sensitive user data via simple URL manipulation. Additionally, missing security headers and improper inventory management leave the system vulnerable to interception and unmanaged attack surfaces. These flaws pose significant regulatory and reputational risks. Immediate remediation through data masking and header hardening is required, alongside a long-term shift toward an API Gateway architecture to ensure a resilient, "Security by Design" environment.

ASSESSMENT OVERVIEW

Objectives

The primary goal of this assessment is to evaluate the security posture of the target API and identify potential vulnerabilities that could lead to unauthorized data access or service disruption. Specific objectives include:

- **Risk Identification**: Pinpoint weaknesses in authentication, authorization, and data exposure.
- **Impact Analysis**: Determine the potential business consequences of identified risks (e.g., data breaches, regulatory non-compliance).
- **Remediation Guidance**: Provide actionable, prioritized recommendations to strengthen the API's defense mechanisms.
- **Skill Validation**: Demonstrate proficiency in modern SaaS security auditing and reporting.

Scope

This assessment is strictly limited to the public demo endpoints provided by JSONPlaceholder. The evaluation focuses on documentation-based analysis through the inspection of headers and response payloads. The testing scope is restricted to read-only requests (GET) and non-intrusive traffic analysis, ensuring that the security posture was evaluated without altering data or disrupting service availability.

Limitations

- **Read-Only/Safe-Mode**: The assessment is non-intrusive. No exploitation, Denial of Service (DoS), or brute-force attempts were performed.
- **Public Environment**: As the target is a public demo environment, backend configurations (e.g., server logs, database settings) were not accessible for review.
- **Time-Boxed**: Findings represent a snapshot in time and may not include vulnerabilities emerging from future code changes.

Methodology

This assessment follows a structured, industry-standard approach based on the OWASP API Security Top 10 (2023) project. The methodology ensures that the analysis covers the most critical risks facing modern web services. The process involved:

1. **Reconnaissance**: Reviewing API documentation to understand endpoint structures and expected data formats.
2. **Traffic Analysis**: Monitoring request/response cycles to inspect headers, status codes, and payload structures.
3. **Vulnerability Mapping**: Comparing observed behaviors against OWASP categories such as Broken Object Level Authorization (BOLA), Unrestricted Resource Consumption, and Broken Authentication.
4. **Risk Rating**: Classifying findings based on the likelihood of discovery and the severity of the business impact.

Tools Used

TOOL	PURPOSE
Postman	Used as the primary API client to construct requests, manage environment variables, and automate testing workflows.
OWASP API Security Top 10	Served as the foundational framework for threat modeling and vulnerability classification.

RISK ASSESSMENT OVERVIEW

RISK CRITERIA

SEVERITY LEVEL	DESCRIPTION
High	Critical vulnerabilities that could result in unauthorized access to sensitive data, full system compromise, or complete bypass of security controls. These require immediate remediation.
Medium	Risks that could allow an attacker to gain limited access to data or disrupt specific functionalities. These often involve misconfigurations or lack of best practices that are difficult to exploit but have significant impact.
Low	Minor security weaknesses, such as information disclosure in headers or lack of non-essential security flags. These pose minimal direct risk but should be addressed to follow defense-in-depth principles.

SUMMARY OF FINDINGS

The table below summarizes the technical findings from JSONPlaceHolder fake rest API analyzed with Postman:

ID	NAME	OWASP CATEGORY	SEVERITY	KEY TECHNICAL RISK
01	Missing Security Headers	API8:2023 – Security Misconfiguration	**Medium**	Absence of HSTS and CSP allows for protocol downgrades and injection attacks.
02	PII Exposure	API3:2023 – Broken Object Property Level Authorization	**High**	Unauthenticated access to emails, phone numbers, and GPS coordinates.
03	Broken Object Level Authorization (BOLA)	API1:2023 – Broken Object Level Authorization	**High**	Lack of ownership validation allows users to scrape private data by changing resource IDs.
04	Improper Inventory Management	API9:2023 – Improper Inventory Management	**Medium**	Undocumented nested resources and potential "Shadow APIs" expand the attack surface.

DETAILED RISK ANALYSIS

1. **Missing Security Headers**

Request: GET https://jsonplaceholder.typicode.com/posts

- OWASP Category: API8:2023 – Security Misconfiguration
- Severity: **Medium**

A secure SaaS usually includes:

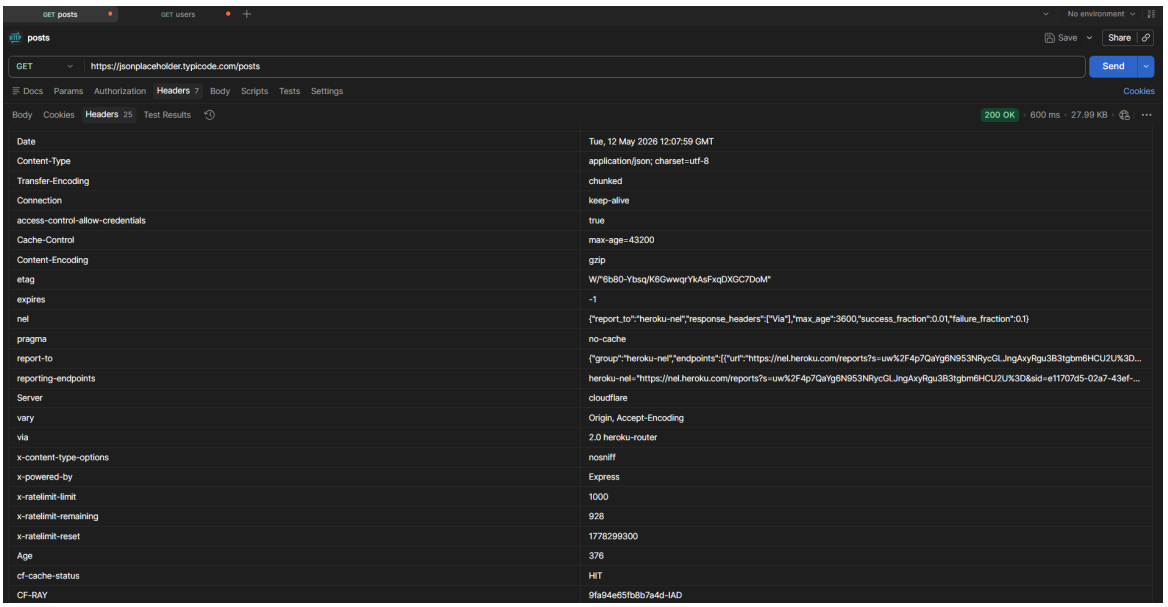
- Strict-Transport-Security (HSTS) – Forces browsers to use HTTPS.
- Content-Security-Policy (CSP) – Prevents various injection attacks.

Technical Analysis

The API's response headers are missing standard security protections, specifically HSTS and CSP. Without HSTS, the server fails to enforce encrypted connections, leaving users open to data interception. Similarly, lacking a CSP means there is no safeguard against malicious code injections or XSS attacks. This oversight ignores Defense-in-Depth best practices, shifting the security burden to the browser rather than maintaining control at the server level.

Business Impact

These misconfigurations risk data interception and session hijacking, potentially compromising sensitive user information. For a SaaS provider, such vulnerabilities damage customer trust and brand reputation. Furthermore, neglecting these headers often leads to failed security audits and non-compliance with standards like GDPR, resulting in legal penalties and lost business opportunities.



2. **PII Exposure via /users Endpoint**

Request: GET https://jsonplaceholder.typicode.com/users

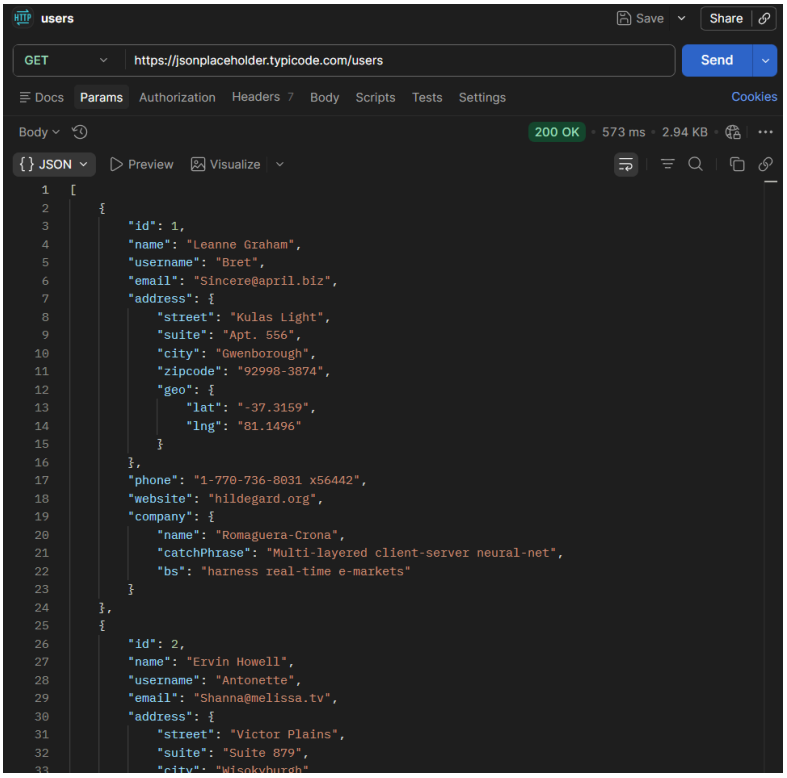
- OWASP Category: API3:2023 – Broken Object Property Level Authorization
- Severity: **High**

Technical Observation

The endpoint returns full user objects including emails, phone numbers, and GPS coordinates without requiring any authentication. This demonstrates a failure to filter sensitive properties at the object level. By exposing internal data fields to unauthorized requesters, the API violates the principle of least privilege and lacks essential data-masking controls.

Business Impact

Publicly leaking PII exposes the company to heavy regulatory fines under GDPR or CCPA and permanent reputational damage. This data allows attackers to launch highly targeted phishing or social engineering campaigns against customers. Ultimately, failing to secure user privacy undermines client trust and can lead to significant legal liabilities.



3. **Broken Object Level Authorization (BOLA)**

Request: GET https://jsonplaceholder.typicode.com/users/1/todos

OWASP Category: API1:2023 - Broken Object Level Authorization

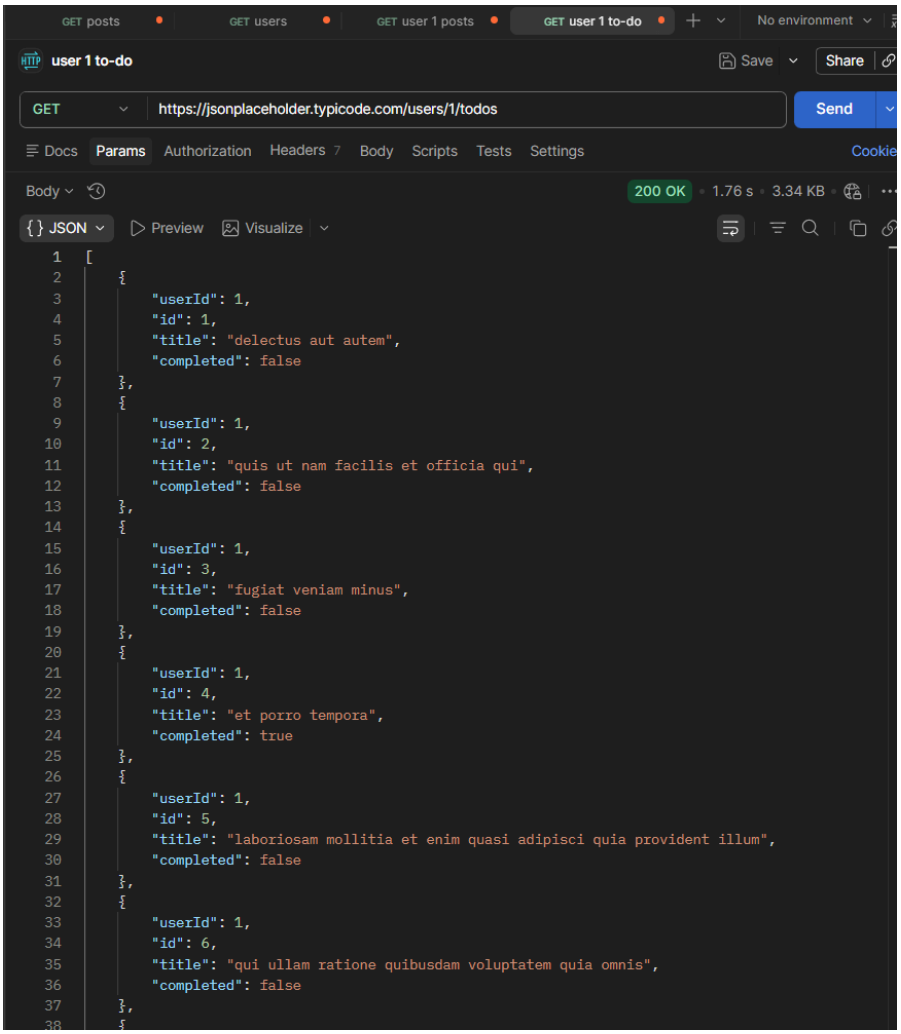
Severity: **High**

Technical Analysis

The API determines which data to return based solely on the ID provided in the URI. Because the server fails to cross-reference the requester's identity with the resource owner, any user can access another's private data by simply changing the URL ID. This allows for trivial automated scraping of the entire database.

Business Impact

This vulnerability leads to unauthorized access to user-specific data, resulting in a massive data breach if internal notes or tasks are exposed. Failing to isolate user records is a primary driver for privacy-related lawsuits and legal penalties. Ultimately, such leaks shatter customer confidence and compromise competitive business intelligence.



4. **Improper Inventory Management / Information Leakage**

Request: GET https://jsonplaceholder.typicode.com/albums/1/photos

OWASP Category: API9:2023 - Improper Inventory Management

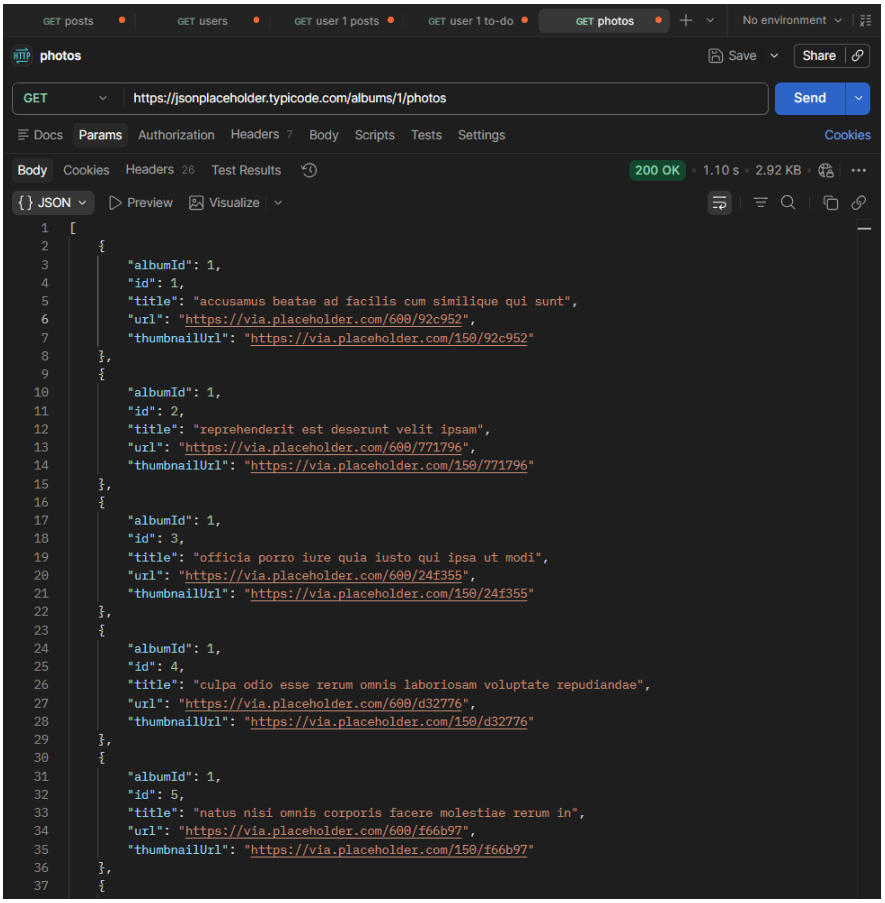
Severity: **Medium**

Technical Analysis

The API supports multiple layers of nested resources without publicly available documentation or version control visibility. This lack of inventory management often hides "Shadow APIs" (old or debug versions) that may lack current security patches or rate limiting. Attackers use automated fuzzing to discover these unmonitored endpoints, creating unmanaged backdoors into the system.

Business Impact

Unmanaged or "zombie" endpoints significantly expand the attack surface, providing extra entry points for data theft. Maintaining undocumented versions also builds technical debt, making it difficult to ensure consistent security across the environment. This complexity often leads to overlooked vulnerabilities, resulting in costly breaches and inefficient use of security resources.



REMEDIATION & STRATEGIC RECOMMENDATIONS

To address the vulnerabilities identified in this assessment, the following actions are recommended. These are categorized by their implementation timeline and impact on the overall security posture.

Short-Term Remediation (Immediate Fixes)

- **Implement Header Hardening**: Deploy security-focused middleware (such as helmet for Node.js) to immediately inject HSTS and CSP headers into all API responses.
- **Enforce Data Masking**: Update the controller logic for the /users endpoint to exclude sensitive fields like geo coordinates and private phone numbers from the public JSON response.
- **Apply Object-Level Validation**: Modify the backend to ensure that the User ID associated with a session token matches the User ID requested in the URI before returning "Todo" data.

Long-Term Strategic Recommendations

- **Deploy an API Gateway**: Transition to an API Gateway architecture (e.g., Kong, AWS API Gateway, or Apigee). This centralizes authentication, logging, and rate-limiting, reducing the risk of Improper Inventory Management.
- **Adopt "Security as Code"**: Integrate automated API security testing into the CI/CD pipeline. This ensures that new endpoints are automatically scanned for BOLA and PII exposure before they reach production.
- **Centralized Identity Management**: Move away from local user management and integrate a robust Identity Provider (IdP) using OAuth 2.0 / OpenID Connect to handle complex authentication flows securely.

CONCLUSION

This security assessment of the JSONPlaceholder environment, while conducted on an open-source tool designed for prototyping, reveals a combination of configuration oversights and fundamental logic flaws. Although the service provides efficient data delivery for testing, the absence of robust authentication and object-level authorization illustrates the critical risks present when security is not integrated into the development lifecycle.

The discovery of high-severity vulnerabilities like BOLA and PII leakage highlights a frequent vulnerability for organizations where accessibility is prioritized over data integrity. For a real-world business, such gaps would lead to severe regulatory penalties and a total loss of customer trust. Implementing short-term remediations, specifically header hardening and strict output filtering, is essential to reducing an API's attack surface.

Ultimately, transitioning toward an API Gateway-centric architecture and adopting a "Security by Design" philosophy is vital for any organization. These steps ensure that as an API scales, it remains resilient, protecting professional reputation and sensitive user information from the start.